A Hybrid Architecture for 2D Game Development: The Case of Asteroid Miner

Luis Alejandro Morales
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
lamoralesn@udistrital.edu.co

Juan Manuel Otálora Hernández
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
jmotalorah@udistrital.edu.co

Julian Felipe Donoso
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
jfdonosoh@udistrital.edu.co

***Abstract*—The development of efficient decision-making systems is a fundamental challenge in modern video game design, particularly in resource management and path optimization scenarios. This paper presents a hybrid architecture for a 2D game called Asteroid Miner, combining greedy algorithms for asteroid selection, backtracking techniques for optimal route planning under fuel constraints, and C++ data structures for managing route history and asteroid catalogs. The proposed approach enables efficient resource maximization while maintaining real-time performance, demonstrating the effectiveness of integrating multiple algorithmic paradigms within an interactive game environment.**

***Index Terms*—Project guidelines, IEEE LaTeX, engineering education, templates**

I. Introduction

The design of intelligent systems for video games has become increasingly relevant as modern applications demand efficient decision-making under constrained environments. In particular, games involving resource management and navigation require algorithms capable of balancing optimality and computational efficiency. These challenges are commonly addressed using classical algorithmic paradigms such as greedy methods and backtracking techniques.

Greedy algorithms are widely used in scenarios where locally optimal decisions can lead to satisfactory global solutions, especially in real-time environments due to their low computational cost. On the other hand, backtracking provides a systematic way to explore multiple possible solutions, making it suitable for problems with constraints such as limited resources. Additionally, efficient data structures such as lists and binary search trees (BST) play a crucial role in organizing and retrieving information within dynamic systems.

This paper presents the design and implementation of *Asteroid Miner*, a 2D game developed using Python and C++, where different algorithmic strategies are integrated into a unified architecture. The system uses a greedy approach to prioritize asteroid mining based on value-distance ratio, while a backtracking algorithm determines optimal routes under fuel constraints. Furthermore, C++ is used to manage route history through lists and to implement an asteroid catalog using a binary search tree.

The main contribution of this work is the integration of multiple computational techniques within an interactive environment, demonstrating how algorithmic theory can be applied to solve practical optimization problems in game development. [**?**].

II. Methods and Materials

A. Problem Formulation

The problem addressed in this work can be formulated as a constrained optimization problem. Given a set of asteroids $A = \{a_1, a_2, ..., a_n\}$, where each asteroid a_i has an associated value v_i and a position (x_i, y_i) in a two-dimensional space, the objective is to determine a sequence of asteroids to visit such that the total collected value is maximized.

The mining unit starts from a fixed base and has a limited fuel capacity. The fuel consumption is proportional to the Euclidean distance traveled between asteroids. Therefore, any feasible solution must satisfy the constraint that the total travel distance does not exceed the available fuel and that the unit is able to return safely to the base.

B. Game Model and Logic

The system is implemented as a 2D game consisting of multiple levels with increasing difficulty. Each level defines a different spatial distribution of asteroids and a fixed fuel budget. The player must select a sequence of asteroids to mine, aiming to maximize resource collection while ensuring a safe return to the base.

Mining an asteroid consumes both time and fuel, reinforcing the need for efficient route planning. At the end of each level, the player's performance is evaluated based on the total resources collected and whether the player successfully returns to the base.

A scoring system based on efficiency is implemented using the optimal solution as a reference. Players achieving results equivalent to the optimal solution receive three stars. Those achieving at least 75% of the optimal value receive two stars, while those meeting a minimum threshold receive one star. Failure to return to the base or to meet the minimum resource requirement results in zero stars.

C. System Architecture

The proposed system follows a hybrid architecture combining Python and C++ components. The Python module is responsible for game logic, algorithm execution, and visualization using Pygame. The C++ module manages data structures for efficient storage and retrieval.

Communication between both components is handled through JSON files, allowing a clear separation of concerns and enabling modular system design. This approach facilitates scalability and simplifies debugging and testing processes.

D. Algorithmic Approach

Algorithm 1 Game Execution Flow

Input: Set of asteroids A, fuel F
Output: Player score
Initialize player at base
while $fuel > 0$ and player selects asteroids **do**
  Move to selected asteroid
  Update fuel based on distance
  Mine asteroid and collect value
end while
Return to base
if player reaches base **then**
  Evaluate collected resources
  Assign stars based on performance
else
  Assign 0 stars
end if

Algorithm 2 Greedy Asteroid Selection

Input: Set of asteroids A, position p
Output: Ordered list of asteroids
for each asteroid $a_i \in A$ **do**
  Compute $r_i = \frac{value_i}{distance(p,a_i)}$
end for
Sort asteroids in descending order of r_i
Return sorted list

E. Data Structures and Implementation

C++ is used to implement efficient data structures required by the system. A list structure is used to store the history of visited asteroids, enabling efficient traversal and reconstruction of routes. Additionally, a binary search tree (BST) is used to maintain an ordered catalog of asteroids based on their value, allowing efficient insertion, search, and retrieval operations [2].

The graphical interface is developed using Pygame, which provides tools for rendering the 2D environment, handling user input, and updating the game state in real time. This integration allows visualization of the decision-making process and facilitates validation of the implemented algorithms.

Although the proposed system is effective for static environments, it assumes that asteroid positions remain constant during gameplay, which may limit its applicability in dynamic scenarios.

Algorithm 3 Optimal Route using Backtracking

Input: Set of asteroids A, fuel F
Output: Best route
$best_value \leftarrow 0$
$best_route \leftarrow \emptyset$
Function Explore(route, remaining_fuel, value)
if $value > best_value$ **then**
  $best_value \leftarrow value$
  $best_route \leftarrow route$
end if
for each asteroid $a_i \notin route$ **do**
  $cost \leftarrow distance(current_position, a_i)$
  if $remaining_fuel - cost \geq 0$ **then**
    Explore($route \cup \{a_i\}$, remaining_fuel - cost, value + v_i)
  end if
end for
Explore($\emptyset$, F, 0)
return $best_route$

Fig. 1. Prototype interface of Asteroid Miner showing asteroid distribution and player route.

III. Results & Discussion

The preliminary results demonstrate clear differences between the implemented algorithmic approaches in terms of efficiency and optimization. The greedy algorithm, based on the value-distance relationship, provides fast and computationally efficient solutions. In most test scenarios, it generates a valid route almost instantaneously, making it suitable for real-time decision-making within the game environment.

However, the greedy approach does not always achieve maximum resource collection. This is particularly true in more complex asteroid distributions, where optimal decisions require considering future movements and fuel constraints.

On the other hand, the backtracking algorithm guarantees the optimal solution by exhaustively exploring all feasible routes under the fuel constraint. This ensures maximum resource collection and a safe return to base. However, this approach has a significantly higher computational cost, especially as the number of asteroids increases, making it less suitable

for real-time execution, but very valuable as a benchmark for evaluating other strategies.

Although they work differently and produce different results, we see that both algorithms will be very useful for the development of our game and its efficiency.

IV. Conclusions

This work demonstrates the feasibility of integrating multiple algorithmic paradigms in a game environment, highlighting the trade-off between optimality and computational efficiency.

Acknowledgments

Optional: funding sources, collaborators who contributed to your work.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 1998.

[3] Pygame Developers, *Pygame Documentation*. [Online]. Available: https://www.pygame.org/docs/

[4] D. Crockford, *The application/json Media Type for JavaScript Object Notation (JSON)*. RFC 8259, 2017.